

Exercise 1: Singleton

```
week_1 > Design Patterns & Principles > Exercise1.java > ...
1  import java.util.*;
2  class Logger
3  {
4      private static Logger Instance;
5      private Logger()
6      {
7          System.out.println(x: "Logger Instance Created");
8      }
9      public static Logger getInstance()
10     {
11         if(Instance == null)
12         {
13             Instance = new Logger();
14         }
15         return Instance;
16     }
17     public void log(String message)
18     {
19         System.out.println("Log :"+message);
20     }
21 }
22 public class Exercise1
23 {
24     Run | Debug
25     public static void main(String[] args) {
26         Logger l1 = Logger.getInstance();
27         Logger l2 = Logger.getInstance();
28         l1.log(message: "first msg by l1");
29         l2.log(message: "first msg by l2");
30         System.out.println(l1==l2);
31     }
32 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
code_5235270b\bin' 'Exercise1'
Logger Instance Created
Log :first msg by l1
Log :first msg by l2
true
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```

Exercise 2: Factory Method Pattern:

```
J Exercise2.java > ...
1  import java.util.*;
2  interface Document {
3      void open();
4  }
5  class WordDocument implements Document{
6      public void open()
7      {
8          System.out.println(x: "Opening Word Document");
9      }
10 }
11 class ExcelDocument implements Document{
12     public void open()
13     {
14         System.out.println(x: "Opening Excel-sheet Document");
15     }
16 }
17 class PDFDocument implements Document{
18     public void open()
19     {
20         System.out.println(x: "Opening PDF-Document");
21     }
22 }
23 abstract class DocumentFactory{
24     public abstract Document create();
25 }
26 class WordCreate extends DocumentFactory
27 {
28     public Document create()
29     {
30         return new WordDocument();
31     }
32 }
33 class ExcelCreate extends DocumentFactory
34 {
35     public Document create(){
36         return new ExcelDocument();
37     }
38 }
39 class PdfCreate extends DocumentFactory
40 {
41     public Document create()
42     {
43         return new PDFDocument();
44     }
45 }
```

```
46 public class Exercise2 {
47     Run | Debug
48     public static void main(String[] args) {
49         DocumentFactory word = new WordCreate();
50         Document ww = word.create();
51         ww.open();
52
53         DocumentFactory excelF = new ExcelCreate();
54         Document Excel = excelF.create();
55         Excel.open();
56
57         DocumentFactory pdfF = new PdfCreate();
58         Document Pdf = pdfF.create();
59         Pdf.open();
60     }
61 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> & 'C:\code_5235270b\bin' 'Exercise2'
Opening Word Document
Opening Excel-sheet Document
Opening PDF-Document
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```

EXERICSE 3: Builder Pattern

```
J Exercise1.java 1 J Exercise2.java 1 J Exercise3.java 1 X
J Exercise3.java > Exercise3 > main(String[])
1 import java.util.*;
2 class Computer
3 {
4     private String cpu;
5     private String ram;
6     private String storage;
7     private String gpu;
8     private String os;
9
10    private Computer(Builder build)
11    {
12        this.cpu = build.cpu;
13        this.ram = build.ram;
14        this.storage = build.storage;
15        this.gpu = build.gpu;
16        this.os = build.os;
17    }
18    public void display()
19    {
20        System.out.println("CPU: "+cpu);
21        System.out.println("RAM: "+ram);
22        System.out.println("Storage: "+storage);
23        System.out.println("GPU: "+gpu);
24        System.out.println("OS: "+os);
25    }
26    public static class Builder{
27        private String cpu;
28        private String ram;
29        private String storage;
30        private String gpu;
31        private String os;
32        public Builder setCpu(String cpu)
33        {
34            this.cpu = cpu;
35            return this;
36        }
37        public Builder setRam(String ram)
38        {
39            this.ram = ram;
40            return this;
41        }
42        public Builder setStorage(String storage)
43        {
44            this.storage = storage;
45            return this;
46        }
47        public Builder setGpu(String gpu)
48        {
49            this.gpu = gpu;
50            return this;
51        }
52        public Builder setOs(String os)
53        {
54            this.os = os;
55            return this;
56        }
57        public Computer build()
58        {
59            return new Computer(this);
60        }
61    }
62 }
```

```
J Exercise3.java > Computer
63 public class Exercise3 {
64     Run | Debug
65     public static void main(String[] args) {
66         Computer gamingpc = new Computer.Builder()
67             .setCpu(cpu: "Intel i9")
68             .setGpu(gpu: "RTX 3000")
69             .setOs(os: "windows")
70             .setRam(ram: "32GB")
71             .setStorage(storage: "1TB SSD").build();
72         Computer personalPC = new Computer.Builder()
73             .setCpu(cpu: "Intel i5")
74             .setOs(os: "Windows")
75             .setRam(ram: "16 GB")
76             .setStorage(storage: "512 GB")
77             .build();
78         System.out.println(x: "Gaming Pc configuration");
79         gamingpc.display();
80
81         System.out.println(x: "PersonalPc configuration");
82         personalPC.display();
83     }
84 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> &
User\workspaceStorage\4c5651b237673004f7d95081d4089c4f\redhat.java\jdt_ws\
Gaming Pc configuration
CPU: Intel i9
RAM: 32GB
Storage: 1TB SSD
GPU: RTX 3000
OS: windows
PersonalPc configuration
CPU: Intel i5
RAM: 16 GB
Storage: 512 GB
GPU: null
OS: Windows
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```

EXERCISE 4: Adaptor design pattern

```
Exercise4.java > Exercise4 > main(String[] args)
1  import java.util.*;
2  interface PaymentProcessor {
3      void processPayment(double amount);
4  }
5  class PayPalGateway{
6      public void sendPayment(double amount)
7      {
8          System.out.println("Payment of Rs."+amount+" processed through paypal");
9      }
10 }
11 class StripeGateway{
12     public void makePayment(double amount)
13     {
14         System.out.println("Payment of Rs."+amount+" processed through StripeGateway");
15     }
16 }
17 class RazorpayGateway{
18     public void payAmount(double amount)
19     {
20         System.out.println("Payment of Rs."+amount+" processed through RazorpayGateway");
21     }
22 }
23 class PayPalAdaptor implements PaymentProcessor{
24     private PayPalGateway pay;
25     PayPalAdaptor(PayPalGateway pay)
26     {
27         this.pay = pay;
28     }
29     public void processPayment(double amount)
30     {
31         pay.sendPayment(amount);
32     }
33 }
34 class StripeAdaptor implements PaymentProcessor{
35     private StripeGateway pay;
36     StripeAdaptor(StripeGateway pay)
37     {
38         this.pay = pay;
39     }
40     public void processPayment(double amount)
41     {
42         pay.makePayment(amount);
43     }
44 }
45 class RazorpayAdaptor implements PaymentProcessor{
46     private RazorpayGateway pay;
47     RazorpayAdaptor(RazorpayGateway pay)
48     {
49         this.pay = pay;
50     }
51     public void processPayment(double amount)
52     {
53         pay.payAmount(amount);
54     }
55 }
```

```
C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code\week_1\Design
Patterns & Principles\Exercise1.java • 1 problem in this file
56 public class Exercise4 {
57     public static void main(String[] args) {
58         PaymentProcessor paypal = new PayPalAdaptor(new PayPalGateway());
59         paypal.processPayment(amount: 1500);
60
61         PaymentProcessor stripe = new StripeAdaptor(new StripeGateway());
62         stripe.processPayment(amount: 2000);
63
64         PaymentProcessor razor = new RazorpayAdaptor(new RazorpayGateway());
65         razor.processPayment(amount: 5000);
66     }
67 }
68

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> & 'C:\Program F
vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d4089c4f\redhat
Payment of Rs.1500.0 processed through paypal
Payment of Rs.2000.0 processed through StripeGateway
Payment of Rs.5000.0 processed through RazorpayGateway
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```


EXERCISE 5: Decorator Design Pattern

```
week_1 > Design Patterns & Principles > Exercise5.java > Exercise5 > main(String[])
1  import java.util.*;
2  interface Notifier{
3      void send();
4  }
5  class EmailNotifier implements Notifier
6  {
7      public void send()
8      {
9          System.out.println(x: "Sending notification via Email");
10     }
11 }
12 abstract class NotifierDecorator implements Notifier{
13     protected Notifier notifier;
14     NotifierDecorator(Notifier notifier)
15     {
16         this.notifier = notifier;
17     }
18     public void send()
19     {
20         notifier.send();
21     }
22 }
23 class SMSNotifierDecorator extends NotifierDecorator{
24     SMSNotifierDecorator(Notifier notifier){
25         super(notifier);
26     }
27     public void send()
28     {
29         super.send();
30         System.out.println(x: "Sending notification via SMS");
31     }
32 }
33 class SlackNotifierDecorator extends NotifierDecorator{
34     SlackNotifierDecorator(Notifier not)
35     {
36         super(not);
37     }
38     public void send()
39     {
40         super.send();
41         System.out.println(x: "Sending notification via Slack");
42     }
43 }
```

```
44 public class Exercises {
45     Run | Debug
46     public static void main(String[] args) {
47         System.out.println(x: "Email Notification:");
48         Notifier email = new EmailNotifier();
49         email.send();
50         System.out.println();
51
52         System.out.println(x: "Email via SMS");
53         Notifier sms = new SMSNotifierDecorator(new EmailNotifier());
54         sms.send();
55         System.out.println();
56
57         System.out.println(x: "msg via email + sms + Slack");
58         // Notifier slack = new SlackNotifierDecorator(new SMSNotifierDecorator(new EmailNotifier()));
59         Notifier all = new SlackNotifierDecorator(sms);
60         all.send();
61     }
62 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> & 'C:\Program Files\Java\jdk-17\bin\java.
vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d4089c4f\redhat.java\jdt_ws\vs_code_52352
Email Notification:
Sending notification via Email

Email via SMS
Sending notification via Email
Sending notification via SMS

msg via email + sms + Slack
Sending notification via Email
Sending notification via SMS
Sending notification via Slack
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```

EXERCISE 6: Proxy Design Pattern

```
week_1 > Design Patterns & Principles > Exercise6.java > ProxyImage > ProxyImage(String)
1  import java.util.*;
2  interface Image{
3      void display();
4  }
5  class RealImage implements Image{
6      private String fileName;
7      RealImage (String fileName)
8      {
9          this.fileName = fileName;
10         loadFromServer();
11     }
12     private void loadFromServer()
13     {
14         System.out.println("Loading "+fileName+" from remote server...");
15     }
16     public void display()
17     {
18         System.out.println("Displaying "+fileName);
19     }
20 }
21 class ProxyImage implements Image{
22     private String fileName;
23     private RealImage image;
24     ProxyImage(String fileName)
25     {
26         this.fileName= fileName;
27     }
28     public void display()
29     {
30         if(image==null)
31             image = new RealImage(fileName);
32         image.display();
33     }
34 }
35 }
```

```
36  public class Exercise6 {
37      Run | Debug
38      public static void main(String[] args) {
39          Image image =new ProxyImage(fileName: "img_2026/06/24.jpg");
40          System.out.println(x: "First Display:");
41          image.display();
42
43          System.out.println();
44          System.out.println(x: "Second Display:");
45          image.display();
46      }
47  }
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> & 'C:\vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d4089c
First Display:
Loading img_2026/06/24.jpg from remote server...
Displaying img_2026/06/24.jpg

Second Display:
Displaying img_2026/06/24.jpg
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> |
```

EXERCISE 7: Observer Design Pattern

```
week_1 > Design Patterns & Principles > J Exercise7.java > Exercise7 > main(String[])
1  import java.util.*;
2  interface Observer{
3      void update(String stockName,double price);
4  }
5  interface Stock{
6      void registerObserver(Observer observer);
7      void removeObserver(Observer observer);
8      void notifyObserver();
9  }
10 class StockMarket implements Stock{
11     private List<Observer> observers = new ArrayList<>();
12     private String stockName;
13     private double price;
14
15     public void setStock(String stockName,double price){
16         this.stockName = stockName;
17         this.price = price;
18         notifyObserver();
19     }
20     public void registerObserver(Observer obs)
21     {
22         observers.add(obs);
23     }
24     public void removeObserver(Observer obs)
25     {
26         observers.remove(obs);
27     }
28     public void notifyObserver(){
29         for(Observer obs:observers)
30         {
31             obs.update(stockName,price);
32         }
33     }
34 }
35 class MobileApp implements Observer{
36     private String user;
37     public MobileApp(String user){
38         this.user = user;
39     }
40     public void update(String stockName, double price)
41     {
42         System.out.println("Mobile App ["+user+"] : "+stockName+" price changed to Rs."+price);
43     }
44 }
45 class WebApp implements Observer{
46     private String user;
47     WebApp(String user){
48         this.user = user;
49     }
50     public void update(String stockName,double price){
51         System.out.println("Web App ["+user+"] : "+stockName+" price changed to Rs."+price);
52     }
53 }
54
```

```

55 public class Exercise7 {
    Run | Debug
56     public static void main(String args[])
57     {
58         StockMarket market = new StockMarket();
59         Observer mobile = new MobileApp(user: "Krishna");
60         Observer web = new WebApp(user: "Admin");
61
62         market.registerObserver(mobile);
63         market.registerObserver(web);
64
65         System.out.println(x: "First Update:");
66         market.setStock(stockName: "STC", price: 4000);
67         System.out.println();
68
69         System.out.println(x: "Second Update:");
70         market.setStock(stockName: "fosys", price: 1500);
71     }
72 }

```

PROBLEMS 6

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> &
vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d
First Update:
Mobile App [Krishna] : STC price changed to Rs.4000.0
Web App [Admin] : STC price changed to Rs.4000.0

Second Update:
Mobile App [Krishna] : fosys price changed to Rs.1500.0
Web App [Admin] : fosys price changed to Rs.1500.0
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>

```


EXERCISE 8: Strategy Design Pattern:

```
week_1 > Design Patterns & Principles > Exercise8.java > Exercise8 > main(String[])
1  import java.util.*;
2  interface PaymentStrategy {
3      void pay(double amount);
4  }
5  class CreditCardPayment implements PaymentStrategy{
6      public void pay(double amount){
7          System.out.println("Paid Rs."+amount+" using Credit card");
8      }
9  }
10 class PaypalPayment implements PaymentStrategy {
11     public void pay(double amount){
12         System.out.println("Paid Rs."+amount+" using Paypal");
13     }
14 }
15 class PaymentContext{
16     private PaymentStrategy strategy;
17     public PaymentContext(PaymentStrategy strategy){
18         this.strategy = strategy;
19     }
20     public void setStrategy(PaymentStrategy strategy){
21         this.strategy = strategy;
22     }
23     public void executePayment(double amount){
24         strategy.pay(amount);
25     }
26 }
```

```
27 public class Exercise8 {
28     Run | Debug
29     public static void main(String[] args) {
30         PaymentContext context = new PaymentContext(new CreditCardPayment());
31         context.executePayment(amount: 1500);
32         context.setStrategy(new PaypalPayment());
33         context.executePayment(amount: 500);
34     }
35 }
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> & 'C:\Program F
vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d4089c4f\redhat
Paid Rs.1500.0 using Credit card
Paid Rs.500.0 using Paypal
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```

EXERISE 9: Command Design Pattern:

```
week_1 > Design Patterns & Principles > J Exercise9.java > Exercise9
1  import java.rmi.Remote;
2
3  interface Command {
4      void execute();
5  }
6
7  class Light {
8      public void turnOn(){
9          System.out.println(x: "Light is ON");
10     }
11     public void turnOff(){
12         System.out.println(x: "Light is Off");
13     }
14 }
15 class LightOncommand implements Command{
16     private Light light;
17
18     LightOncommand(Light light){
19         this.light = light;
20     }
21     public void execute(){
22         light.turnOn();
23     }
24 }
25 class LightOffcommand implements Command{
26     private Light light;
27
28     LightOffcommand(Light light){
29         this.light = light;
30     }
31     public void execute(){
32         light.turnOff();
33     }
34 }
35 class RemoteControl {
36     private Command cmd;
37
38     public void setCommand(Command cmd){
39         this.cmd = cmd;
40     }
41     public void pressButton(){
42         cmd.execute();
43     }
44 }
```

```
44 public class Exercise9 {  
    Run | Debug  
45     public static void main(String[] args) {  
46         Light li = new Light();  
47         Command cmdOn = new LightOncommand(li);  
48         Command cmdOff = new LightOffcommand(li);  
49  
50         RemoteControl remote = new RemoteControl();  
51         remote.setCommand(cmdOn);  
52         remote.pressButton();  
53  
54         remote.setCommand(cmdOff);  
55         remote.pressButton();  
56     }  
57 }
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant  
User\workspaceStorage\4c5651b237673004f7d95081d4089c4f\redhat.j  
Light is ON  
Light is Off
```

EXERCISE 10: Architectural pattern – MVC

```
week_1 > Design Patterns & Principles > J Exercise10.java > Exercise10 > main(String[])
1  class Student{
2      private String name;
3      private int id;
4      private String grade;
5
6      Student(String name,int id, String grade){
7          this.name = name;
8          this.id = id;
9          this.grade = grade;
10     }
11     public String getName(){
12         return name;
13     }
14     public void setName(String name){
15         this.name = name;
16     }
17     public int getId(){
18         return id;
19     }
20     public void setId(int id){
21         this.id = id;
22     }
23     public String getGrade(){
24         return grade;
25     }
26     public void setGrade(String grade){
27         this.grade = grade;
28     }
29 }
30 class StudentView{
31     public void displayStudentDetails(Student student){
32         System.out.println("Student Details");
33         System.out.println("Name: "+student.getName());
34         System.out.println("ID: "+student.getId());
35         System.out.println("Grade: "+student.getGrade());
36     }
37 }
38 class StudentController{
39     private Student model;
40     private StudentView view;
41
42     public StudentController(Student model,StudentView view){
43         this.model = model;
44         this.view = view;
45     }
46     public void setStudentName(String name){
47         model.setName(name);
48     }
49     public void setStudentGrade(String grade){
50         model.setGrade(grade);
51     }
52     public void updateView(){
53         view.displayStudentDetails(model);
54     }
55 }
```


Exercise 11: Dependency Injection

```
week_1 > Design Patterns & Principles > J Exercise11.java > ...
1  interface CustomerRepository{
2      String findCustomerById(int id);
3  }
4  class CustomerRepositoryImpl implements CustomerRepository{
5      public String findCustomerById(int id){
6          return "Customer ID: "+id+", Name: Krishna Vishwa";
7      }
8  }
9  class CustomerService{
10     private CustomerRepository repository;
11
12     public CustomerService(CustomerRepository repo){
13         repository = repo;
14     }
15     public void getCustomer(int id){
16         System.out.println(repository.findCustomerById(id));
17     }
18 }
19 public class Exercise11 {
    Run | Debug
20     public static void main(String[] args){
21         CustomerRepository repo = new CustomerRepositoryImpl();
22         CustomerService serv = new CustomerService(repo);
23         serv.getCustomer(id: 101);
24     }
25 }

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code> &
vishwa\AppData\Roaming\Code\User\workspaceStorage\4c5651b237673004f7d95081d4
Customer ID: 101, Name: Krishna Vishwa
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\vs_code>
```